

A Mathematical Note on `dm_cmb.ipynb`: Patch-Based CMB Prior Learning with Flow Matching

Technical note for the diffusion-model CMB notebook

May 25, 2026

Abstract

This note documents the mathematical content of the notebook `dm_cmb.ipynb`, whose current purpose is to learn a generative prior for CMB T, Q, U patches at 143 GHz and to validate that prior through flat-sky spectral diagnostics. The workflow combines standard CMB harmonic methods, including spin-weighted polarization decompositions and Gaussian simulations from C_ℓ spectra [1–4], with a modern path-based generative model trained by direct flow matching [8–10]. We describe the full pipeline from Planck half-ring preprocessing and simulated prior construction to gnomonic patch extraction, train/validation normalization, probability-path definition, ODE sampling, training losses, and spectral validation. Particular attention is paid to the debugging history of the notebook, because many of the implemented modifications—denoising before spectral estimation, direct vector-field prediction, low-frequency temperature losses, variance calibration, and spectral checkpoint selection—are scientifically meaningful responses to identifiable failure modes rather than arbitrary tuning. The aim is to provide a self-contained methods note that can serve both as documentation and as the starting point for a more formal paper.

Contents

1	Introduction	3
2	CMB Power Spectra	3
2.1	From C_ℓ to the plotted quantity D_ℓ	5
2.2	Why a flat-sky approximation is reasonable for the patch diagnostics	6
3	Observed Data and Why It Is Prepared	7
3.1	Planck half-ring maps	7
3.2	Planck preprocessing	8
4	Simulated Priors and Why They Are Needed	10
4.1	Foreground prior $p(x)$ from PySM	10
4.2	CMB prior $p(f)$ from CAMB + Healpy	11
4.2.1	CAMB spectra	11
4.2.2	Gaussian T, Q, U map simulation	11
4.3	Why both priors exist	12
5	Patch Extraction, Polarization Rotation, and Normalization	12
5.1	Gnomonic patches	12
5.2	Polarization rotation	13
5.3	Dataset splitting and normalization	13

6	Probability Paths	14
6.1	General Gaussian conditional path	14
6.2	Schedule options implemented	15
7	ODE/SDE Samplers	16
7.1	Original score-based parameterization	16
7.2	Current direct flow-matching sampler	16
7.3	Heun integrator	17
8	Reconstruction Maps	18
8.1	From epsilon prediction	18
8.2	From vector-field prediction	18
9	U-Net Architecture	19
9.1	Time embedding	19
9.2	Residual blocks	19
9.3	Encoder / decoder	19
10	Training Formulations and Why They Changed	20
10.1	Stage 1: epsilon prediction	20
10.2	Stage 2: direct flow matching	20
10.3	Stage 3: auxiliary clean-map reconstruction loss	20
10.4	Stage 4: low-frequency T -channel loss	21
10.5	Stage 5: smoothed variance matching loss	22
10.6	Current total loss	22
10.7	Principled objective versus calibration terms	22
10.8	Ramped and decaying auxiliary losses	23
11	Timestep Sampling Bias	24
12	Optimization and EMA	24
13	Flat-Sky Spectral Diagnostics	24
13.1	Power spectrum estimator	24
13.2	Smoothing before spectra	26
13.3	Why x_0 -hat spectra are used	26
13.4	Variance-matched diagnostic	26
14	Diagnostic Quantities Added During Debugging	27
15	Chronology of Major Fixes and What They Solved	27
15.1	Fix 1: true validation split	27
15.2	Fix 2: clip fraction and endpoint range	27
15.3	Fix 3: denoise before spectra	27
15.4	Fix 4: direct flow matching	28
15.5	Fix 5: EMA for sampling, then EMA warmup	28
15.6	Fix 6: low- t bias and x_0 reconstruction loss	28
15.7	Fix 7: low-frequency T loss	28
15.8	Fix 8: smoothed variance matching loss	28
15.9	Fix 9: staged calibration and spectral checkpoint selection	28
16	Current Active Configuration	28

17 Discussion: Interpretation of the Current Remaining Error	29
17.1 A more systematic way to “hit the right spot”	29
17.2 How to interpret future overshoot or undershoot	30
18 Conclusion	31

1 Introduction

We expand on previous work using deep learning methods to perform CMP component separation (cite) and further explore the use of Denoising Probabilistic Diffusion Models (DPDM) to perform component separation between true CMB signals and foreground noise (dust and synchrotron emission). Let y = observed data, x = foreground signal, f = true CMB, ϕ = lensing potential, n = noise and A = instrumental effects (beam and sky mask) transformation. Then the observed signal can be expressed as

$$y = A\mathcal{L}(\phi)(f + x) + n \quad (1)$$

where $\mathcal{L}$ is the lensing map. We want to sample the posterior

$$P(x, f|y) \sim P(y|x, f)P(x)P(f) \quad (2)$$

To do so we need the priors $P(x)$ and $P(f)$ which will serve as our training data. We train two separate U-Net models to learn vector fields u_t^θ that correctly generate probability paths from initial Gaussian distributions to the priors distributions, ODE/SDE samplers. Finally, we sample the posterior likelihood we are interested in. This is the step where the true CMB data enters, via the likelihood gradients $\nabla_x \log(y|x, f)$ and $\nabla_f \log(y|x, f)$.

We perform our analysis for a single frequency (143 GHz) using Planck PR4 half ring maps and masking maps as our source of observed CMB sky. We use CAMB to generate the theoretical CMB spectrum for both temperature and polarization on the sphere for given cosmological parameters, assuming Gaussianity. Finally, we use PySM to generate realistic foreground emission maps (dust and synchrotron emission).

2 CMB Power Spectra

The CMB temperature anisotropy is a scalar field on the sphere and is therefore naturally expanded in spherical harmonics [1, 3],

$$T(\hat{n}) = \sum_{\ell=0}^{\infty} \sum_{m=-\ell}^{\ell} a_{\ell m}^T Y_{\ell m}(\hat{n}). \quad (3)$$

From the orthonormality of the spherical-harmonics

$$\int_{\mathbb{S}^2} Y_{\ell m}(\hat{n}) Y_{\ell' m'}^*(\hat{n}) d\Omega = \delta_{\ell\ell'} \delta_{mm'}, \quad (4)$$

the harmonic coefficients are obtained by

$$a_{\ell m}^T = \int_{\mathbb{S}^2} T(\hat{n}) Y_{\ell m}^*(\hat{n}) d\Omega. \quad (5)$$

Thus $a_{\ell m}^T$ is the amplitude with which the angular pattern appears in the sky. In a statistically isotropic theory, this basis is particularly useful because two-point statistics become diagonal in ℓ and m . The multipole ℓ controls angular scale, with the rough correspondence

$$\theta \sim \frac{\pi}{\ell}, \quad (6)$$

so small ℓ corresponds to large coherent angular structures and large ℓ to fine structure. The azimuthal index m resolves the angular dependence within a fixed ℓ .

Linear polarization is defined as **EXPAND**

$$(Q \pm iU)(\hat{n}) = \sum_{\ell m} a_{\ell m}^{\pm 2} {}_{\pm 2}Y_{\ell m}(\hat{n}). \quad (7)$$

Their coefficients are likewise obtained by projection:

$$a_{\ell m}^{\pm 2} = \int_{\mathbb{S}^2} (Q \pm iU)(\hat{n}) {}_{\pm 2}Y_{\ell m}^*(\hat{n}) d\Omega. \quad (8)$$

The scalar E - and pseudo-scalar B -mode coefficients are then defined by [1, 2]

$$a_{\ell m}^E = -\frac{1}{2} (a_{\ell m}^2 + a_{\ell m}^{-2}), \quad a_{\ell m}^B = \frac{i}{2} (a_{\ell m}^2 - a_{\ell m}^{-2}). \quad (9)$$

Equivalently, one may think of E as the curl-free part of polarization and B as the divergence-free part. In standard parity-conserving Λ CDM, the primary nonzero two-point spectra are

$$C_{\ell}^{TT}, \quad C_{\ell}^{EE}, \quad C_{\ell}^{BB}, \quad C_{\ell}^{TE}, \quad (10)$$

while TB and EB vanish in the absence of parity-violating physics.

For a statistically isotropic Gaussian CMB, the harmonic coefficients satisfy

$$\mathbb{E}[a_{\ell m}^X (a_{\ell' m'}^Y)^*] = C_{\ell}^{XY} \delta_{\ell\ell'} \delta_{mm'}, \quad X, Y \in \{T, E, B\}. \quad (11)$$

Thus all statistical information is contained in the covariance matrix at each ℓ :

$$\Sigma_{\ell} = \begin{pmatrix} C_{\ell}^{TT} & C_{\ell}^{TE} & 0 \\ C_{\ell}^{TE} & C_{\ell}^{EE} & 0 \\ 0 & 0 & C_{\ell}^{BB} \end{pmatrix}. \quad (12)$$

This is why the notebook evaluates generated samples through TT, EE, BB, TE : if the prior is intended to model Gaussian CMB structure faithfully, then matching these spectra is the most direct second-order validation test.

Equivalently, one may say that for a Gaussian isotropic sky the entire prior is determined by the family of covariance matrices Σ_ℓ . Once the notebook has the correct Σ_ℓ for all relevant scales, it has captured the correct second-order structure of the Gaussian CMB prior.

The same statement can be derived from the real-space covariance function. For temperature,

$$\xi_T(\hat{n}, \hat{n}') := \mathbb{E}[T(\hat{n})T(\hat{n}')] \quad (13)$$

$$= \mathbb{E} \left[\sum_{\ell m} a_{\ell m}^T Y_{\ell m}(\hat{n}) \sum_{\ell' m'} (a_{\ell' m'}^T)^* Y_{\ell' m'}^*(\hat{n}') \right] \quad (14)$$

$$= \sum_{\ell m} \sum_{\ell' m'} \mathbb{E}[a_{\ell m}^T (a_{\ell' m'}^T)^*] Y_{\ell m}(\hat{n}) Y_{\ell' m'}^*(\hat{n}'). \quad (15)$$

Using isotropy,

$$\mathbb{E}[a_{\ell m}^T (a_{\ell' m'}^T)^*] = C_\ell^{TT} \delta_{\ell\ell'} \delta_{mm'}, \quad (16)$$

so only equal ℓ, m terms survive:

$$\xi_T(\hat{n}, \hat{n}') = \sum_{\ell=0}^{\infty} C_\ell^{TT} \sum_{m=-\ell}^{\ell} Y_{\ell m}(\hat{n}) Y_{\ell m}^*(\hat{n}'). \quad (17)$$

Applying the spherical-harmonic addition theorem,

$$\sum_{m=-\ell}^{\ell} Y_{\ell m}(\hat{n}) Y_{\ell m}^*(\hat{n}') = \frac{2\ell+1}{4\pi} P_\ell(\hat{n} \cdot \hat{n}'), \quad (18)$$

one obtains

$$\xi_T(\hat{n}, \hat{n}') = \sum_{\ell=0}^{\infty} \frac{2\ell+1}{4\pi} C_\ell^{TT} P_\ell(\hat{n} \cdot \hat{n}'). \quad (19)$$

Thus the full temperature two-point function is determined by $\{C_\ell^{TT}\}$, and the same logic extends to the full T, E, B covariance structure.

2.1 From C_ℓ to the plotted quantity D_ℓ

The notebook often plots

$$D_\ell^{XY} = \frac{\ell(\ell+1)}{2\pi} C_\ell^{XY}. \quad (20)$$

This is only a rescaled version of the same information, but it is often visually more informative because for nearly scale-invariant spectra the factor $\ell(\ell+1)$ roughly removes the leading ℓ^{-2} behavior of C_ℓ and makes acoustic peaks easier to see.

The statement that D_ℓ^{TT} measures temperature variance per logarithmic ℓ -interval comes from the variance formula. Starting from the two-point function above and setting $\hat{n}' = \hat{n}$, one has $P_\ell(1) = 1$, hence

$$\text{Var}[T(\hat{n})] = \sum_{\ell=0}^{\infty} \frac{2\ell+1}{4\pi} C_\ell^{TT}. \quad (21)$$

This is the exact full-sky identity. To obtain the logarithmic- ℓ interpretation, approximate the sum at moderate and large ℓ by an integral:

$$\sum_{\ell} f(\ell) \approx \int f(\ell) d\ell, \quad (22)$$

and use $2\ell + 1 \approx 2\ell$. Then

$$\text{Var}[T(\hat{n})] \approx \int \frac{\ell C_\ell^{TT}}{2\pi} d\ell. \quad (23)$$

Now write $d\ell = \ell d \ln \ell$, which yields

$$\text{Var}[T(\hat{n})] \approx \int \frac{\ell^2 C_\ell^{TT}}{2\pi} d \ln \ell. \quad (24)$$

At large enough ℓ , one may replace the sum by an integral and use $2\ell + 1 \approx 2\ell$:

$$\frac{\ell^2 C_\ell^{TT}}{2\pi} \approx \frac{\ell(\ell + 1) C_\ell^{TT}}{2\pi} = D_\ell^{TT}, \quad (25)$$

so

$$\text{Var}[T(\hat{n})] \approx \int D_\ell^{TT} d \ln \ell. \quad (26)$$

Thus D_ℓ^{TT} is, up to the standard large- ℓ approximation, the contribution to temperature variance per unit logarithmic multipole interval. The same interpretation applies to D_ℓ^{EE} and D_ℓ^{BB} . For the cross-spectrum D_ℓ^{TE} , the sign matters and the quantity measures covariance rather than variance.

In particular:

- D_ℓ^{TT} measures temperature variance per logarithmic ℓ -interval;
- D_ℓ^{EE} and D_ℓ^{BB} measure polarization variance per logarithmic ℓ -interval for the parity-even and parity-odd sectors;
- D_ℓ^{TE} measures temperature–polarization covariance per logarithmic ℓ -interval and may be positive or negative.

2.2 Why a flat-sky approximation is reasonable for the patch diagnostics

The training objects are 64×64 tangent-plane patches at $6.7'$ per pixel, so each patch spans

$$64 \times 6.7' \approx 429' \approx 7.15^\circ \quad (27)$$

across. On such small areas, the sphere is well approximated by a plane and spherical harmonics are locally approximated by Fourier modes:

$$Y_{\ell m}(\hat{n}) \longrightarrow e^{i\boldsymbol{\ell} \cdot \mathbf{x}}. \quad (28)$$

Therefore the notebook estimates spectra from 2D Fourier transforms of the patches rather than from full-sky harmonic transforms. The price is that these are *flat-sky* spectral diagnostics, but the gain is that they are directly matched to the patch-based training domain.

More precisely, on a sufficiently small patch the spherical metric differs from the Euclidean one only by curvature corrections of order θ^2 , where θ is the patch radius. In that regime, the spherical Laplacian eigenvalue equation

$$-\nabla_{\mathbb{S}^2}^2 Y_{\ell m} = \ell(\ell + 1) Y_{\ell m} \quad (29)$$

is locally approximated by the planar relation

$$-\nabla_{\mathbb{R}^2}^2 e^{i\boldsymbol{\ell} \cdot \mathbf{x}} = |\boldsymbol{\ell}|^2 e^{i\boldsymbol{\ell} \cdot \mathbf{x}}, \quad (30)$$

so that $|\boldsymbol{\ell}|^2$ plays the role of $\ell(\ell + 1)$. This is the mathematical basis for interpreting the local Fourier wavevector $\boldsymbol{\ell}$ as the flat-sky analogue of the spherical multipole.

One may make the approximation more concrete as follows. Fix a small neighborhood of a sky direction $\hat{n}_0$ and introduce tangent-plane coordinates $\mathbf{x} = (x, y)$ via the gnomonic projection used in the notebook. A scalar sky field is then represented locally by a planar field $T_{\text{patch}}(\mathbf{x})$. Its Fourier transform is

$$\tilde{T}(\boldsymbol{\ell}) = \int_{\mathbb{R}^2} T_{\text{patch}}(\mathbf{x}) e^{-i\boldsymbol{\ell}\cdot\mathbf{x}} d^2\mathbf{x}, \quad (31)$$

and for a statistically homogeneous flat-sky field one defines

$$\mathbb{E}[\tilde{T}(\boldsymbol{\ell}) \tilde{T}^*(\boldsymbol{\ell}')] = (2\pi)^2 \delta^{(2)}(\boldsymbol{\ell} - \boldsymbol{\ell}') C_{\boldsymbol{\ell}}^{TT, \text{flat}}, \quad \ell = |\boldsymbol{\ell}|. \quad (32)$$

The isotropic flat-sky power spectrum therefore depends only on the radial frequency $\ell = |\boldsymbol{\ell}|$, exactly as the full-sky spectrum depends only on the multipole ℓ .

On a finite rectangular patch of angular side lengths L_x, L_y , the Fourier modes become discrete:

$$\ell_x = \frac{2\pi k_x}{L_x}, \quad \ell_y = \frac{2\pi k_y}{L_y}, \quad k_x, k_y \in \mathbb{Z}. \quad (33)$$

The notebook computes FFTs of the projected T, Q, U patches, constructs flat-sky E/B fields from the Fourier-space Q/U modes, and then averages $|\tilde{X}(\boldsymbol{\ell})|^2$ or $\tilde{X}(\boldsymbol{\ell})\tilde{Y}^*(\boldsymbol{\ell})$ over annuli in $|\boldsymbol{\ell}|$. This annular averaging is the flat-sky analogue of averaging over all m at fixed full-sky multipole ℓ .

The flat-sky polarization formulas can also be derived explicitly. Define the complex polarization field

$$P(\mathbf{x}) := Q(\mathbf{x}) + iU(\mathbf{x}). \quad (34)$$

For a spin-2 field on the plane, the Fourier-space decomposition is

$$P(\mathbf{x}) = - \int \frac{d^2\boldsymbol{\ell}}{(2\pi)^2} [E(\boldsymbol{\ell}) + iB(\boldsymbol{\ell})] e^{2i\phi_{\boldsymbol{\ell}}} e^{i\boldsymbol{\ell}\cdot\mathbf{x}}, \quad (35)$$

with $\phi_{\boldsymbol{\ell}} = \arg(\boldsymbol{\ell})$. Taking Fourier transforms gives

$$Q_{\boldsymbol{\ell}} + iU_{\boldsymbol{\ell}} = - [E_{\boldsymbol{\ell}} + iB_{\boldsymbol{\ell}}] e^{2i\phi_{\boldsymbol{\ell}}}. \quad (36)$$

Multiplying by $e^{-2i\phi_{\boldsymbol{\ell}}}$ and separating real and imaginary parts yields

$$E_{\boldsymbol{\ell}} = -Q_{\boldsymbol{\ell}} \cos(2\phi_{\boldsymbol{\ell}}) - U_{\boldsymbol{\ell}} \sin(2\phi_{\boldsymbol{\ell}}), \quad (37)$$

$$B_{\boldsymbol{\ell}} = Q_{\boldsymbol{\ell}} \sin(2\phi_{\boldsymbol{\ell}}) - U_{\boldsymbol{\ell}} \cos(2\phi_{\boldsymbol{\ell}}), \quad (38)$$

which are exactly the formulas implemented later in the notebook.

The approximation is useful because the model is trained on local projected patches, not on the full sphere. The diagnostics should therefore live in the same geometry as the learning problem. Using full-sky $a_{\ell m}$ estimators inside the training loop would mix together projection effects, masking, and patch-boundary issues that are absent from the model input space. The flat-sky approximation avoids that mismatch and is quantitatively reasonable because the patches are small compared with the curvature scale of the sphere.

3 Observed Data and Why It Is Prepared

3.1 Planck half-ring maps

The notebook imports two Planck PR4/NPIPE half-ring maps at 143 GHz, denoted here by

$$h_1, \quad h_2 \in \mathbb{R}^{3 \times N_{\text{pix}}}, \quad (39)$$

with channels (T, Q, U) , together with a sky mask. The discussion here follows the standard Planck mapmaking and beam/pixel conventions used for public CMB frequency maps [3, 5].

These maps are not used as direct training targets for the prior model. Instead, they serve two purposes:

1. to define the observational object

$$y = \frac{h_1 + h_2}{2}, \quad (40)$$

which is the approximate observed sky map;

2. to define the half-ring difference proxy

$$d = \frac{h_1 - h_2}{2}, \quad (41)$$

which acts as a noise proxy / validation object for later likelihood-based work.

3.2 Planck preprocessing

Each half-ring map is preprocessed by `preprocess_pr4_hr_to_nside512`:

1. read (T, Q, U) and infer or set the input unit;
2. convert to μK_{CMB} ;
3. replace invalid / unseen pixels by zero;
4. smooth harmonically from an assumed input beam of $7.27'$ FWHM to an output beam of $10'$ FWHM;
5. downgrade to $N_{\text{side}} = 512$;
6. use $l_{\text{max}} = 3N_{\text{side}} - 1 = 1535$.

The $7.27'$ in step 4 is the *assumed effective input beam width in the current preprocessing code*. It is not the same thing as the $6.7'$ patch pixel size used later for tangent-plane projections. These two numbers refer to different operations:

- the beam FWHM quantifies physical angular smoothing, i.e. the width of the point-spread function;
- the $6.7'$ patch pixel size quantifies numerical sampling on the projected flat patch.

Thus the report should indeed distinguish them. In the notebook as currently written, the harmonic preprocessing assumes an input beam of $7.27'$ and adds only the extra smoothing needed to reach an output beam of $10'$. The later $6.7'$ number describes the projected patch grid and is not an input beam parameter.

If one instead asks for the *native or effective angular resolution of the Planck 143 GHz channel itself*, then the relevant number is different again: the official Planck beam summaries quote an effective FWHM of roughly $7.27'$ for 143 GHz frequency maps, i.e. a value close to $7'$, not $5'$. Therefore there are three distinct angular scales in play:

1. the physical 143 GHz map beam, of order $7.2'$;
2. the notebook's current simplifying assumption $fwhm_{\text{in}} = 7.27'$ inside `preprocess_pr4_hr_to_nside512`;
3. the projected patch sampling $6.7'$ per pixel.

Only item 1 is a property of the Planck data product itself. Items 2 and 3 are analysis choices made by the notebook. The value 7.27' is chosen to be closer to the published effective 143 GHz beam than the earlier 5' placeholder.

In harmonic space, if $a_{\ell m}$ denotes a scalar or spin component, the additional smoothing is

$$a_{\ell m}^{\text{out}} = b_{\ell}^{\text{add}} a_{\ell m}^{\text{in}}, \quad b_{\ell}^{\text{add}} = \exp\left[-\frac{1}{2}\ell(\ell+1)\sigma_{\text{add}}^2\right], \quad (42)$$

with σ_{add} chosen so that

$$\text{FWHM}_{\text{out}}^2 = \text{FWHM}_{\text{in}}^2 + \text{FWHM}_{\text{add}}^2. \quad (43)$$

Equivalently,

$$b_{\ell}^{\text{add}} = \frac{B_{\ell}^{\text{out}}}{B_{\ell}^{\text{in}}}, \quad B_{\ell}^{\text{in/out}} = \exp\left[-\frac{1}{2}\ell(\ell+1)\sigma_{\text{in/out}}^2\right]. \quad (44)$$

So the preprocessing does not create a fresh 10' beam from a delta-function map; it assumes the input already carries a 7.27' effective beam and then multiplies by the incremental ratio needed to reach 10'.

After beam smoothing and pixelization, the effective observed harmonic coefficients are of the form

$$\tilde{a}_{\ell m}^X = B_{\ell}^X P_{\ell}^X a_{\ell m}^X, \quad (45)$$

where B_{ℓ} is the instrumental or imposed Gaussian beam and P_{ℓ} is the HEALPix pixel window. Working at a common output beam and common N_{side} ensures that observed data, simulated priors, and later likelihood ingredients live in a harmonically compatible convention.

For an azimuthally symmetric Gaussian beam with full width at half maximum FWHM, the harmonic beam transfer function is

$$B_{\ell} = \exp\left[-\frac{1}{2}\ell(\ell+1)\sigma_b^2\right], \quad \sigma_b = \frac{\text{FWHM}}{\sqrt{8 \ln 2}}. \quad (46)$$

Thus beam smoothing is a low-pass filter in harmonic space: high- ℓ modes are exponentially suppressed more than low- ℓ modes.

On the sky, the same operation can be written as a convolution with an azimuthally symmetric kernel:

$$(B * T)(\hat{n}) = \int_{\mathbb{S}^2} B(\hat{n} \cdot \hat{n}') T(\hat{n}') d\Omega', \quad (47)$$

and the spherical-harmonic basis diagonalizes that convolution, which is why it becomes simple multiplication by B_{ℓ} in harmonic space. This is one of the main reasons beam manipulations are performed on $a_{\ell m}$ rather than directly on pixel maps.

Pixelation is different from beam smoothing. A pixelized map does not store the value of the sky at an exact direction; it stores an average over a finite pixel footprint. If $W_p(\hat{n})$ denotes the window for pixel p , normalized so that $\int W_p(\hat{n}) d\Omega = 1$, then the stored map value is approximately

$$m_p = \int_{\mathbb{S}^2} W_p(\hat{n}) T(\hat{n}) d\Omega. \quad (48)$$

For HEALPix, this finite-area averaging induces an additional low-pass response, summarized approximately by the isotropic pixel window P_{ℓ} . In harmonic language, beam smoothing and pixel averaging therefore combine multiplicatively:

$$\tilde{a}_{\ell m}^X = B_{\ell}^X P_{\ell}^X a_{\ell m}^X. \quad (49)$$

The beam describes optical or imposed smoothing of the underlying sky, whereas the pixel window describes averaging caused by finite spatial sampling.

This distinction matters operationally. If one downgrades to a coarser N_{side} without sufficient prior smoothing, then small-scale power that cannot be represented on the coarser grid aliases into lower multipoles. Smoothing to a common $10'$ beam before downgrade reduces that unresolved high- ℓ power and makes the subsequent $N_{\text{side}} = 512$ representation stable. It also makes observed maps and simulated priors comparable: both are expressed with the same effective angular resolution and the same pixelization convention.

If a sky mask $M(\hat{n})$ is applied, the observed field becomes

$$T^{\text{obs}}(\hat{n}) = M(\hat{n}) T(\hat{n}), \quad (50)$$

and similarly for Q, U . Multiplication in real space becomes mode coupling in harmonic space:

$$a_{\ell m}^{\text{obs}} = \sum_{\ell' m'} K_{\ell m, \ell' m'} a_{\ell' m'}, \quad (51)$$

with coupling kernel K determined by the mask. This is another reason the notebook works with valid projected patches rather than attempting a delicate full-sky pseudo- C_ℓ treatment inside the training loop.

4 Simulated Priors and Why They Are Needed

4.1 Foreground prior $p(x)$ from PySM

The foreground prior is constructed from PySM 3 simulations at 143 GHz, using dust and synchrotron presets [6, 7]:

$$\text{fg_presets} = (\text{d1}, \text{s1}). \quad (52)$$

For each foreground realization:

1. generate a base full-sky (T, Q, U) map;
2. convert to μK_{CMB} ;
3. optionally apply a random Euler rotation (α, β, γ) on the sphere;
4. smooth to the same $10'$ beam convention as the CMB prior;
5. extract many local gnomonic patches.

If the simulated foreground map is denoted $x(\hat{n})$, then the random rotation step conceptually applies an element $R \in SO(3)$ to produce

$$x_R(\hat{n}) = x(R^{-1}\hat{n}). \quad (53)$$

In spherical-harmonic language, the corresponding scalar coefficients would transform as

$$a'_{\ell m} = \sum_{m'=-\ell}^{\ell} D_{mm'}^{\ell}(R) a_{\ell m'}, \quad (54)$$

with $D_{mm'}^{\ell}$ the Wigner D -matrices; for polarization, the same rotation acts on the underlying spin-2 field. The purpose is not to alter the physical foreground statistics, but to decorrelate the training set from one preferred sky orientation.

The unit-conversion step can be summarized schematically as

$$x_{\mu\text{K}_{\text{CMB}}}(\hat{n}, \nu) = g_{\nu} x_{\text{native}}(\hat{n}, \nu), \quad (55)$$

where g_{ν} is the frequency-dependent conversion from the PySM native brightness convention to thermodynamic CMB temperature units. This is crucial because the subsequent training mixes temperature and polarization numerically and requires a common physical unit convention.

The random rotation is used so that the network does not overfit specific sky orientations or large-scale alignments of a single foreground realization.

4.2 CMB prior $p(f)$ from CAMB + Healpy

The true-CMB prior used for the current main training run is a Gaussian CMB prior generated from C_ℓ spectra computed with CAMB [4].

4.2.1 CAMB spectra

The notebook constructs a standard Planck-like cosmology through `CAMBparams`, with default values close to

$$H_0 = 67.5, \quad \Omega_b h^2 = 0.022, \quad \Omega_c h^2 = 0.122, \quad \tau = 0.06, \quad n_s = 0.965, \quad A_s = 2.1 \times 10^{-9}, \quad (56)$$

and then extracts the raw spectra

$$C_\ell^{TT}, \quad C_\ell^{EE}, \quad C_\ell^{BB}, \quad C_\ell^{TE}. \quad (57)$$

The helper

$$D_\ell = \frac{\ell(\ell+1)}{2\pi} C_\ell \quad (58)$$

is used for plotting, but the map simulation is performed from C_ℓ , not D_ℓ .

4.2.2 Gaussian T, Q, U map simulation

The function `simulate_cmb_tqu_from_cls` generates Gaussian full-sky CMB maps by:

1. setting the unphysical low multipoles $\ell = 0, 1$ to zero;
2. drawing correlated Gaussian harmonic coefficients

$$(a_{\ell m}^T, a_{\ell m}^E, a_{\ell m}^B) \quad (59)$$

with covariance implied by TT, EE, BB, TE ;

3. multiplying by a Gaussian beam B_ℓ with $10'$ FWHM;
4. multiplying by the HEALPix pixel window P_ℓ (temperature and polarization separately);
5. transforming back to (T, Q, U) on an $N_{\text{side}} = 512$ grid.

More explicitly, for each $\ell \geq 2$ and each m , the harmonic triplet is drawn from a zero-mean complex Gaussian law with covariance Σ_ℓ :

$$\begin{pmatrix} a_{\ell m}^T \\ a_{\ell m}^E \\ a_{\ell m}^B \end{pmatrix} \sim \mathcal{N}_{\mathbb{C}}(0, \Sigma_\ell), \quad \Sigma_\ell = \begin{pmatrix} C_\ell^{TT} & C_\ell^{TE} & 0 \\ C_\ell^{TE} & C_\ell^{EE} & 0 \\ 0 & 0 & C_\ell^{BB} \end{pmatrix}, \quad (60)$$

subject to the reality condition

$$a_{\ell, -m}^X = (-1)^m (a_{\ell m}^X)^*. \quad (61)$$

Once $a_{\ell m}^T, a_{\ell m}^E, a_{\ell m}^B$ are generated, the beam and pixel window are applied:

$$\bar{a}_{\ell m}^X = B_\ell^X P_\ell^X a_{\ell m}^X. \quad (62)$$

The final Q, U maps are then synthesized from $\bar{a}_{\ell m}^E, \bar{a}_{\ell m}^B$ using the spin-2 inverse harmonic transform. This is why the simulated prior is Gaussian by construction but still has the correct TE correlation and the correct polarization mode content.

Operationally, drawing the correlated Gaussian harmonic coefficients means choosing, for each ℓ , a matrix square root L_ℓ such that

$$\Sigma_\ell = L_\ell L_\ell^\dagger, \quad (63)$$

sampling a standard complex Gaussian vector $\mathbf{g}_{\ell m}$, and setting

$$\mathbf{a}_{\ell m} = L_\ell \mathbf{g}_{\ell m}. \quad (64)$$

Then

$$\mathbb{E}[\mathbf{a}_{\ell m} \mathbf{a}_{\ell m}^\dagger] = L_\ell \mathbb{E}[\mathbf{g}_{\ell m} \mathbf{g}_{\ell m}^\dagger] L_\ell^\dagger = L_\ell L_\ell^\dagger = \Sigma_\ell. \quad (65)$$

So the simulator is explicitly building harmonic-space random variables with the desired covariance and then transforming them back to the map domain.

This gives Gaussian CMB skies consistent with the chosen spectra and instrumental smoothing convention.

4.3 Why both priors exist

The notebook infrastructure builds both:

- $p(f)$: true CMB prior (`prior_f`),
- $p(x)$: foreground prior (`prior_x`).

However, the current active training run focuses on the true CMB prior only:

$$\text{path.p_data} = \text{PriorSampler}(\text{prior_f}, \text{subset_fraction}=1). \quad (66)$$

5 Patch Extraction, Polarization Rotation, and Normalization

5.1 Gnomonic patches

The prior maps are converted from full-sky HEALPix format to flat gnomonic patches:

- patch size 64×64 ,
- pixel size $6.7'$,
- random centers chosen within the sky mask.

If a footprint mask is provided, the notebook can require every pixel in the projected patch to satisfy

$$m(\hat{n}) \geq 0.9. \quad (67)$$

Mathematically, if (θ_0, ϕ_0) is the patch center and (ξ, η) are local tangent-plane coordinates, the gnomonic projection maps nearby directions on the sphere to a plane tangent at (θ_0, ϕ_0) . One convenient forward form is

$$\xi = \frac{\cos \theta \sin(\phi - \phi_0)}{\sin \theta_0 \sin \theta + \cos \theta_0 \cos \theta \cos(\phi - \phi_0)}, \quad (68)$$

$$\eta = \frac{\cos \theta_0 \sin \theta - \sin \theta_0 \cos \theta \cos(\phi - \phi_0)}{\sin \theta_0 \sin \theta + \cos \theta_0 \cos \theta \cos(\phi - \phi_0)}. \quad (69)$$

On sufficiently small patches, this projection is close to a Cartesian coordinate chart. That is exactly why local FFTs are appropriate for subsequent spectral diagnostics.

The full prior-construction map therefore consists of a sequence of transformations:

$$\text{harmonic prior} \xrightarrow{\text{beam/pixel convention}} \text{full-sky } (T, Q, U) \xrightarrow{\text{mask / allowed centers}} \text{selected sky locations} \xrightarrow{\text{gnomonic projection}} \text{gnomonic projection} \quad (70)$$

The requirement that the whole projected footprint lie in a valid region is a nonlinear selection step designed to prevent edge contamination from entering the FFT-based diagnostics.

5.2 Polarization rotation

For (Q, U) , a local spin-2 basis rotation is applied when projecting to the gnomonic tangent plane. If ψ is the local basis rotation angle, then

$$\begin{pmatrix} Q' \\ U' \end{pmatrix} = \begin{pmatrix} \cos 2\psi & \sin 2\psi \\ -\sin 2\psi & \cos 2\psi \end{pmatrix} \begin{pmatrix} Q \\ U \end{pmatrix}. \quad (71)$$

Equivalently, in complex notation,

$$Q' \pm iU' = e^{\mp 2i\psi} (Q \pm iU). \quad (72)$$

The factor of 2 is the defining signature of a spin-2 field. If this rotation were omitted, the projected polarization would not be expressed in a consistent local basis, and the inferred flat-sky E/B modes would be contaminated by a purely geometric mismatch.

This is important because the native HEALPix polarization basis is not the same as the local Cartesian patch basis.

5.3 Dataset splitting and normalization

After patch extraction, each prior dataset is shuffled deterministically, split into train/validation sets, and normalized channel-wise using *train-only* statistics:

$$\mu_c = \frac{1}{N_{\text{train}} HW} \sum_{n,i,j} x_{n,c,i,j}, \quad \sigma_c = \sqrt{\frac{1}{N_{\text{train}} HW} \sum_{n,i,j} (x_{n,c,i,j} - \mu_c)^2}. \quad (73)$$

The normalized patches are

$$\tilde{x}_{n,c,i,j} = \frac{x_{n,c,i,j} - \mu_c}{\sigma_c}. \quad (74)$$

If the total shuffled patch collection is

$$\mathcal{D} = \{x_{\pi(1)}, \dots, x_{\pi(N)}\}, \quad (75)$$

for a fixed permutation π , then the split is

$$\mathcal{D}_{\text{train}} = \{x_{\pi(1)}, \dots, x_{\pi(N_{\text{train}})}\}, \quad \mathcal{D}_{\text{val}} = \{x_{\pi(N_{\text{train}}+1)}, \dots, x_{\pi(N)}\}. \quad (76)$$

The important statistical rule is that the normalization constants μ_c, σ_c are computed only from $\mathcal{D}_{\text{train}}$, so validation remains genuinely held out.

Because the averages are taken over all training patches and all pixels within each channel, the normalization is estimating the empirical first and second moments of the patch prior:

$$\mu_c \approx \mathbb{E}[x_c], \quad \sigma_c^2 \approx \text{Var}(x_c). \quad (77)$$

This standardization is mathematically simple but numerically important: it makes the channels comparable in scale and turns the training problem into one in which the network mostly has to learn structure rather than trivial unit mismatches.

The currently saved `.npz` priors contain, for both $p(f)$ and $p(x)$:

$$29491 \text{ train patches}, \quad 3277 \text{ validation patches}, \quad (78)$$

each of shape $3 \times 64 \times 64$.

The model is trained in this normalized patch domain.

This normalization is itself a linear transformation of the prior. If $S = \text{diag}(\sigma_T, \sigma_Q, \sigma_U)$, then for each pixel vector $m = (T, Q, U)^T$,

$$\tilde{m} = S^{-1}(m - \mu). \quad (79)$$

Therefore the learned prior is mathematically the prior of the transformed field $\tilde{m}$, not of the raw physical field m . This is harmless for training stability and model comparison because both train and validation sets are transformed in the same way, but it does mean that the notebook’s diagnostic spectra are spectra in the normalized domain unless the normalization is explicitly undone.

For temperature alone, the effect is simple:

$$\tilde{C}_\ell^{TT} = \sigma_T^{-2} C_\ell^{TT}. \quad (80)$$

For polarization it is slightly subtler because E and B are formed *after* separately normalizing Q and U . If $\sigma_Q \neq \sigma_U$, then normalized-domain EE and BB are not just one-number rescalings of the physical spectra. The notebook’s spectra therefore serve primarily as *model-versus-reference diagnostics in the training domain*, not as direct substitutes for physical full-sky C_ℓ values.

There is also an averaging subtlety: the empirical μ_c, σ_c are moments of the *projected patch distribution* induced by the chosen mask, patch centers, beam convention, and projection operator. So the learned prior is the prior of “patches produced by this exact pipeline,” not an abstract full-sky field with no observational operator attached.

6 Probability Paths

6.1 General Gaussian conditional path

The notebook defines a family of Gaussian conditional paths

$$x_t = \alpha_t z + \beta_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad (81)$$

where z is a data sample and α_t, β_t are scalar schedules.

The corresponding conditional score is

$$\nabla_x \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}, \quad (82)$$

and the conditional vector field implemented in the notebook is

$$u_t(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x. \quad (83)$$

These formulas follow directly from the Gaussian conditional law. Since

$$x_t | z \sim \mathcal{N}(\alpha_t z, \beta_t^2 I), \quad (84)$$

its log density is, up to an additive constant,

$$\log p_t(x | z) = -\frac{1}{2\beta_t^2} \|x - \alpha_t z\|^2 - d \log \beta_t, \quad (85)$$

where d is the ambient dimension. Differentiating with respect to x gives

$$\nabla_x \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}. \quad (86)$$

To derive the conditional vector field, differentiate the path $x_t = \alpha_t z + \beta_t \varepsilon$ at fixed z, ε :

$$\frac{dx_t}{dt} = \dot{\alpha}_t z + \dot{\beta}_t \varepsilon. \quad (87)$$

Now solve for ε from the path equation,

$$\varepsilon = \frac{x_t - \alpha_t z}{\beta_t}, \quad (88)$$

and substitute back:

$$u_t(x | z) = \dot{\alpha}_t z + \dot{\beta}_t \frac{x - \alpha_t z}{\beta_t} \quad (89)$$

$$= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x. \quad (90)$$

So the vector field is simply the time derivative of the conditional path written as a function of the current state x instead of the latent noise ε .

It is useful to make the role of this vector field completely explicit. The path variable x_t is the *state actually seen by the network at time t* . It is a mixture of signal and Gaussian noise. The latent noise ε is useful for constructing the path, but it is not observable from x_t alone unless one also knows z . The quantity

$$v_t := \frac{dx_t}{dt} \quad (91)$$

therefore plays a special role: it is the infinitesimal transport direction of the state along the path. In other words, v_t tells us how a sample at time t should move if we evolve it toward the data endpoint.

For the current linear schedule,

$$\alpha_t = t, \quad \beta_t = 1 - t, \quad \dot{\alpha}_t = 1, \quad \dot{\beta}_t = -1, \quad (92)$$

so

$$x_t = tz + (1 - t)\varepsilon, \quad v_t = \dot{x}_t = z - \varepsilon. \quad (93)$$

This identity is especially important: the target vector field is independent of t once z and ε are fixed. Therefore the model is not being asked to learn an arbitrary time-dependent target. It is learning the direction that transports the Gaussian endpoint ε into the data sample z .

This also explains why v_t is closely related to x_t and ε : the pair (x_t, v_t) contains enough information to recover both z and ε , provided the schedule is nondegenerate. That fact is used later in the reconstruction formula.

6.2 Schedule options implemented

Three schedule families are present in the notebook.

Linear schedule

$$\alpha_t = t, \quad \beta_t = 1 - t. \quad (94)$$

This is the *current active training path*.

Exact trigonometric schedule

$$\alpha_t = \sin\left(\frac{\pi t}{2}\right), \quad \beta_t = \cos\left(\frac{\pi t}{2}\right), \quad \alpha_t^2 + \beta_t^2 = 1. \quad (95)$$

This was implemented and tested, but in practice it produced overly flat spectra in the current standardized-data regime, so it is not the active configuration.

Bounded trigonometric schedule This uses

$$\alpha_t = \sin \theta(t), \quad \beta_t = \cos \theta(t), \quad (96)$$

with

$$\theta(t) = \theta_{\min} + (\theta_{\max} - \theta_{\min})t, \quad \theta_{\min} = \arcsin(\alpha_{\min}), \quad \theta_{\max} = \arccos(\beta_{\min}), \quad (97)$$

where historically $\alpha_{\min} = \beta_{\min} = 0.02$ or 0.05 . This was introduced to avoid stiff endpoint behavior in the original score-based sampler.

7 ODE/SDE Samplers

7.1 Original score-based parameterization

The original model predicted noise $\hat{\varepsilon}_\theta(x_t, t)$ and defined an approximate score

$$s_t(x) = -\frac{\hat{\varepsilon}_\theta(x, t)}{\beta_t}. \quad (98)$$

The notebook implemented the score-to-drift conversion

$$u_t(x) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) s_t(x) + \frac{\dot{\alpha}_t}{\alpha_t} x, \quad (99)$$

and then an SDE family

$$dX_t = \left(u_t(X_t) + \frac{\sigma^2}{2} s_t(X_t) \right) dt + \sigma dW_t. \quad (100)$$

For the linear schedule $\alpha_t = t$, $\beta_t = 1 - t$, this drift becomes very stiff near $t = 0$, which caused severe instability and saturation in early experiments.

7.2 Current direct flow-matching sampler

The current model predicts the vector field directly, so the sampler is simply the deterministic ODE

$$\frac{dX_t}{dt} = \hat{v}_\theta(X_t, t). \quad (101)$$

This is the simplest continuous-time generative interpretation of the learned field. If $\hat{v}_\theta$ exactly equaled the true probability-flow field, then solving this ODE from the simple endpoint distribution at $t = 0$ would transport that law to the data law at $t = 1$ [10]. In practice the learned field is approximate, so the notebook uses numerical diagnostics and spectral validation to judge whether the endpoint distribution is close enough to the target prior.

The conceptual difference from ε -prediction is worth spelling out. In the older formulation, the network predicts $\hat{\varepsilon}_\theta$, which must then be converted into a score, then into a drift, and only then integrated. In the current formulation, the network predicts the drift-like quantity directly:

$$\hat{v}_\theta(x, t) \approx v_t(x). \quad (102)$$

So the learned object is already the right-hand side of the ODE. This removes one layer of algebraic conversion and makes the geometry of the sampler much easier to interpret: samples simply flow along the learned vector field from noise to data.

For the linear path, the exact conditional field is

$$v_t(x \mid z) = z - \varepsilon = \frac{z - x_t}{1 - t} = \frac{x_t - \varepsilon}{t}, \quad (103)$$

depending on whether one eliminates ε or z . These equivalent forms show both why low- t errors are influential for endpoint quality and why the direct vector-field formulation is numerically better behaved than the earlier score conversion.

The notebook wraps this as `VectorFieldODE` with zero diffusion. This is numerically integrated with:

- Euler–Maruyama for $\sigma > 0$,
- Heun’s method for $\sigma = 0$ (current active choice).

7.3 Heun integrator

For step size Δt , the current deterministic sampler uses

$$k_1 = \hat{v}_\theta(x_t, t), \quad (104)$$

$$x^\star = x_t + \Delta t k_1, \quad (105)$$

$$k_2 = \hat{v}_\theta(x^\star, t + \Delta t), \quad (106)$$

$$x_{t+\Delta t} = x_t + \frac{\Delta t}{2}(k_1 + k_2). \quad (107)$$

Heun’s method is the explicit trapezoidal rule, or equivalently a two-stage Runge–Kutta method. It can be understood geometrically in three steps:

1. evaluate the vector field at the current point (x_t, t) , giving the initial slope k_1 ;
2. take a provisional Euler step to a predicted next point $x^\star$;
3. re-evaluate the vector field there, obtaining a second slope k_2 , and then advance using the average slope $(k_1 + k_2)/2$.

So Heun is doing a local “predictor–corrector” update. Euler alone would use only k_1 ,

$$x_{t+\Delta t}^{\text{Euler}} = x_t + \Delta t k_1, \quad (108)$$

which is only first-order accurate. Heun instead averages the slope at the start and the slope at the predicted end of the interval, making it second-order accurate for smooth ODEs. In Taylor-series terms, if the true solution satisfies

$$x(t + \Delta t) = x(t) + \Delta t \dot{x}(t) + \frac{\Delta t^2}{2} \ddot{x}(t) + O(\Delta t^3), \quad (109)$$

then Heun captures the $O(\Delta t^2)$ term correctly, whereas Euler does not.

This matters in the notebook because the learned vector field is only approximate. A lower-order integrator would convert modest local vector-field errors into visibly distorted spectra more quickly. Heun reduces that discretization error, especially in the moderate number of integration steps used during training diagnostics and post-training sampling.

One can also interpret Heun statistically. If $\hat{v}_\theta$ is the network’s best local estimate of the transport direction, then Heun asks for that estimate at two nearby points and averages them. This makes the sampler less sensitive to local curvature of the learned flow than a single Euler slope would be.

Historically the notebook also supported a state clamp

$$x \leftarrow \text{clip}(x, -x_{\max}, x_{\max}), \quad (110)$$

with $x_{\max} = 5$, used mainly as a diagnostic guardrail after severe blow-up was observed in the original score-based sampler. The current active runs have essentially zero clipping fraction.

8 Reconstruction Maps

The sampler returns a terminal state x_t , not directly a clean sample z . Therefore the notebook reconstructs $\hat{z}$.

This distinction is crucial. The state x_t is the point reached by the path dynamics at time t . Unless $t = 1$ and the path exactly lands on the data manifold, x_t is still a mixed object containing some amount of residual noise-like component. The reconstruction map $\hat{z}$ is therefore not an arbitrary post-processing step; it is the notebook's estimate of the underlying clean sample that gave rise to the current state under the chosen probability path.

8.1 From epsilon prediction

In the old noise-prediction parameterization,

$$\hat{z} = \frac{x_t - \beta_t \hat{\varepsilon}_\theta(x_t, t)}{\alpha_t}. \quad (111)$$

This is obtained by rearranging the path equation

$$x_t = \alpha_t z + \beta_t \varepsilon. \quad (112)$$

If the network predicts $\hat{\varepsilon}_\theta \approx \varepsilon$, then solving for z gives

$$z \approx \frac{x_t - \beta_t \hat{\varepsilon}_\theta}{\alpha_t}. \quad (113)$$

So even in the epsilon-parameterization, the reconstruction map is simply the algebraic inversion of the path under the network's estimated latent noise.

8.2 From vector-field prediction

In the current flow-matching parameterization, the notebook reconstructs $\hat{z}$ from the 2×2 linear system defined by

$$x_t = \alpha_t z + \beta_t \varepsilon, \quad v_t = \dot{\alpha}_t z + \dot{\beta}_t \varepsilon. \quad (114)$$

Solving for z gives

$$\hat{z} = \frac{\dot{\beta}_t x_t - \beta_t \hat{v}_\theta(x_t, t)}{\alpha_t \dot{\beta}_t - \dot{\alpha}_t \beta_t}. \quad (115)$$

This expression comes from solving the 2×2 linear system

$$\begin{pmatrix} x_t \\ \hat{v}_t \end{pmatrix} = \begin{pmatrix} \alpha_t & \beta_t \\ \dot{\alpha}_t & \dot{\beta}_t \end{pmatrix} \begin{pmatrix} z \\ \varepsilon \end{pmatrix}, \quad (116)$$

where $\hat{v}_t := \hat{v}_\theta(x_t, t)$. Provided the determinant

$$\Delta_t := \alpha_t \dot{\beta}_t - \dot{\alpha}_t \beta_t \quad (117)$$

is nonzero, the inverse matrix is

$$\begin{pmatrix} z \\ \varepsilon \end{pmatrix} = \frac{1}{\Delta_t} \begin{pmatrix} \dot{\beta}_t & -\beta_t \\ -\dot{\alpha}_t & \alpha_t \end{pmatrix} \begin{pmatrix} x_t \\ \hat{v}_t \end{pmatrix}. \quad (118)$$

Reading off the first component yields the formula above for $\hat{z}$. This derivation is useful because it clarifies that the reconstruction is not an ad hoc denoiser; it is the algebraic inversion of the chosen probability path.

For the linear path $\alpha_t = t$, $\beta_t = 1 - t$, this simplifies to

$$\hat{z} = x_t + (1 - t)\hat{v}_\theta(x_t, t). \quad (119)$$

This formula gives a very direct interpretation of what the reconstruction map is doing. If t is close to 1, then x_t is already close to the clean endpoint and only a small correction $(1 - t)\hat{v}_\theta$ is added. If t is smaller, then the state is farther from the data endpoint and the reconstruction uses a larger extrapolation along the learned transport direction to recover $\hat{z}$.

So the reconstruction map $\hat{z}$ should be understood as:

- the notebook’s estimate of the clean T, Q, U patch corresponding to the current noisy state x_t ,
- expressed in the same normalized training domain as the priors,
- and computed by inverting the chosen path with the learned vector field inserted in place of the unknown true one.

If the model and solver were exact, then $\hat{z}$ would equal the true endpoint sample z . In practice $\hat{z}$ is only approximate, but it is the correct object to compare against the training and validation priors because those priors live at the clean-data endpoint of the path, not at intermediate noisy times.

9 U-Net Architecture

The active network is **CMBUNet**, operating on $3 \times 64 \times 64$ normalized patches.

9.1 Time embedding

Time is embedded by a learnable Fourier feature map:

$$\phi(t) = \sqrt{2} \begin{bmatrix} \sin(2\pi t w_1), \dots, \sin(2\pi t w_{d/2}), \\ \cos(2\pi t w_1), \dots, \cos(2\pi t w_{d/2}) \end{bmatrix}, \quad (120)$$

with learnable frequencies w_i .

9.2 Residual blocks

Each residual layer performs:

$$h_1 = \text{Conv}(\text{GN}(\text{SiLU}(x))), \quad (121)$$

adds a time-conditioned bias from the embedding,

$$h_2 = h_1 + A(\phi(t)), \quad (122)$$

then applies another GN–SiLU–Conv block and adds the residual connection.

9.3 Encoder / decoder

The current architecture is:

- channels: $[32, 64, 128, 256]$,
- 2 residual layers per stage,
- time embedding dimension 64,
- downsampling by strided 3×3 convolutions,
- upsampling by bilinear resize followed by 3×3 convolution.

The notebook stores encoder activations and adds them back in the decoder as skip connections.

10 Training Formulations and Why They Changed

10.1 Stage 1: epsilon prediction

The original diffusion-style training objective was

$$L_\varepsilon = \mathbb{E} \left[\|\hat{\varepsilon}_\theta(x_t, t) - \varepsilon\|^2 \right]. \quad (123)$$

This is mathematically natural for noise prediction, but in the current notebook it combined with the score-conversion sampler to produce unstable reverse dynamics, large clip fractions, and severely distorted spectra.

It is also the standard squared-error loss associated with a Gaussian regression model. If one assumes that, conditional on (x_t, t) ,

$$\varepsilon \sim \mathcal{N}(\hat{\varepsilon}_\theta(x_t, t), \sigma_\varepsilon^2 I), \quad (124)$$

then the negative log-likelihood is, up to an additive constant and a multiplicative factor $1/(2\sigma_\varepsilon^2)$,

$$-\log p(\varepsilon \mid x_t, t; \theta) \propto \|\hat{\varepsilon}_\theta(x_t, t) - \varepsilon\|^2. \quad (125)$$

So the notebook’s MSE objective is the maximum-likelihood estimator under a Gaussian conditional error model.

10.2 Stage 2: direct flow matching

The notebook was then rewritten to predict the vector field directly.

For the linear path,

$$x_t = tz + (1 - t)\varepsilon, \quad v^\star = z - \varepsilon. \quad (126)$$

The base flow-matching loss is

$$L_v = \mathbb{E} \left[\|\hat{v}_\theta(x_t, t) - v^\star\|^2 \right]. \quad (127)$$

This removed the stiff score conversion and fixed the main instability problem.

Again, this is a familiar squared-error regression objective. Under the Gaussian conditional model

$$v^\star \sim \mathcal{N}(\hat{v}_\theta(x_t, t), \sigma_v^2 I), \quad (128)$$

minimizing L_v is exactly maximum likelihood. The same logic applies to the later x_0 -reconstruction term: it is the Gaussian regression loss associated with predicting the clean endpoint from the noisy state.

For the current linear path, one can write the target especially simply:

$$x_t = tz + (1 - t)\varepsilon, \quad \frac{dx_t}{dt} = z - \varepsilon. \quad (129)$$

Thus the flow-matching target is constant along the conditional path. This is one reason the linear path was attractive experimentally: it gives a strong and interpretable transport signal from pure Gaussian noise at $t = 0$ to the data point z at $t = 1$.

10.3 Stage 3: auxiliary clean-map reconstruction loss

The notebook then added a reconstruction term

$$L_{x_0} = \mathbb{E} \left[\|\hat{z} - z\|^2 \right]. \quad (130)$$

For the linear path,

$$\hat{z} - z = (1 - t)(\hat{v}_\theta - v^\star), \quad (131)$$

so L_{x_0} is a time-weighted version of vector-field error that emphasizes low t , precisely where endpoint sample quality is most sensitive.

This identity is worth deriving explicitly. For the linear path,

$$\hat{z} = x_t + (1 - t)\hat{v}_\theta, \quad z = x_t + (1 - t)v^\star, \quad (132)$$

because $v^\star = z - \varepsilon$ and $x_t = tz + (1 - t)\varepsilon$. Subtracting gives

$$\hat{z} - z = (1 - t)(\hat{v}_\theta - v^\star). \quad (133)$$

Therefore

$$L_{x_0} = \mathbb{E}[(1 - t)^2 \|\hat{v}_\theta - v^\star\|^2]. \quad (134)$$

So L_{x_0} is not an unrelated extra objective. It is exactly the same vector-field error viewed through the lens of endpoint reconstruction.

The combined loss became

$$L_{\text{flow+recon}} = L_v + \lambda_{x_0} L_{x_0}, \quad (135)$$

with current active value

$$\lambda_{x_0} = 0.25. \quad (136)$$

10.4 Stage 4: low-frequency T -channel loss

When the TT peak location became roughly correct but the low/mid- ℓ amplitude remained too small, the notebook added a low-frequency temperature loss. Let G_σ denote Gaussian blur with width σ pixels, and let $\hat{T}, T$ be the T -channel slices of $\hat{z}$ and z . Then

$$L_{\text{lf},T}^{(\sigma)} = \mathbb{E}[\|G_\sigma * \hat{T} - G_\sigma * T\|^2]. \quad (137)$$

The notebook averages this over

$$\sigma \in \{2, 4\} \text{ pixels}. \quad (138)$$

In Fourier space, Gaussian blur applies the weight

$$\widehat{G_\sigma * T}(k) = e^{-\frac{1}{2}\sigma^2|k|^2} \hat{T}(k), \quad (139)$$

so this loss emphasizes low and mid spatial frequencies. It was added specifically because TT was too low at low/mid ℓ even when the overall shape was becoming correct.

By Parseval’s identity, this can be rewritten as a frequency-weighted MSE:

$$\|G_\sigma * \hat{T} - G_\sigma * T\|^2 \propto \sum_k e^{-\sigma^2|k|^2} |\hat{T}(k) - T(k)|^2. \quad (140)$$

So low- $|k|$ modes receive the largest penalty, while high- $|k|$ mismatches are strongly down-weighted. Because on the flat sky $|k|$ is proportional to ℓ , this is a direct way to emphasize the low- and intermediate- ℓ TT structure that dominates the broad acoustic peak.

Using two blur widths, $\sigma \in \{2, 4\}$ pixels, creates two overlapping low-pass windows:

- $\sigma = 2$ pixels still responds to mid- ℓ temperature structure,
- $\sigma = 4$ pixels pushes more strongly on the largest coherent scales within each patch.

Their average is therefore a smoother and more robust proxy for “match the large-scale TT morphology” than any single filter width would be.

The current active coefficient is

$$\lambda_{\text{lf},T} = 0.5. \quad (141)$$

10.5 Stage 5: smoothed variance matching loss

Later diagnostics showed that the raw generated spectra often looked like under-dispersed versions of the target spectra, while variance-matched diagnostic plots improved substantially. This suggested that the main remaining issue had become multiplicative under-variance.

To target this directly, the notebook now adds a smoothed per-channel variance matching loss. Let

$$\tilde{z} = G_{\sigma_{\text{var}}} * z, \quad \tilde{\hat{z}} = G_{\sigma_{\text{var}}} * \hat{z}, \quad (142)$$

with

$$\sigma_{\text{var}} = 1.2 \text{ pixels}, \quad (143)$$

matching the same smoothing used in spectral evaluation.

For each channel $c \in \{T, Q, U\}$, compute variance across batch and spatial dimensions:

$$v_c^{\text{gen}} = \text{Var}(\tilde{z}_c), \quad v_c^{\text{ref}} = \text{Var}(\tilde{\hat{z}}_c). \quad (144)$$

The current notebook uses the *log-variance* form

$$L_{\text{var}} = \frac{1}{3} \sum_c \left(\log(v_c^{\text{gen}} + \epsilon) - \log(v_c^{\text{ref}} + \epsilon) \right)^2. \quad (145)$$

This is more symmetric than the earlier relative-error form. In particular, overestimating the variance by a factor a and underestimating it by the reciprocal factor a^{-1} now incur the same penalty, because

$$\log(a) - \log(1) = -(\log(a^{-1}) - \log(1)). \quad (146)$$

This is important precisely in the regime reached later in the notebook, where the model can move from under-powered spectra to over-powered spectra.

The current active coefficient is

$$\lambda_{\text{var}} = 1.0. \quad (147)$$

10.6 Current total loss

At full auxiliary strength, the active composite objective is

$$L_{\text{total}} = L_v + \lambda_{x_0} L_{x_0} + \lambda_{\text{lf}, T} L_{\text{lf}, T} + \lambda_{\text{var}} L_{\text{var}}. \quad (148)$$

During actual training, however, the notebook does not use this full-strength form from the very beginning; the next subsection explains the epoch-dependent ramp applied to the spectral-calibration terms.

10.7 Principled objective versus calibration terms

It is useful to distinguish two different notions of “correctness”:

1. the *principled flow-matching objective*,
2. the *practical calibration terms* needed by a finite network and finite optimizer.

For the linear path

$$x_t = tz + (1 - t)\varepsilon, \quad v^* = z - \varepsilon, \quad (149)$$

the theoretically fundamental objective is simply

$$L_v = \mathbb{E} \|\hat{v}_\theta(x_t, t) - v^*\|^2. \quad (150)$$

In the idealized limit of infinite data, infinite model capacity, exact optimization, and exact ODE integration, minimizing L_v alone should recover the correct path marginals and therefore the correct data distribution at $t = 1$.

The additional losses

$$L_{x_0}, \quad L_{\text{lf},T}, \quad L_{\text{var}} \quad (151)$$

are therefore *not* required by the mathematical definition of flow matching itself. They are added because the practical model is not in the ideal regime. Once these terms are present, the training problem becomes explicitly multi-objective, and the hyperparameters $\lambda_{x_0}, \lambda_{\text{lf},T}, \lambda_{\text{var}}$ are calibration parameters rather than uniquely determined theory constants.

This is exactly why the notebook can move from spectra that are too low to spectra that are too high: once the main transport field is approximately correct, the remaining problem is not gross instability but *amplitude calibration*. The correct way to think about this stage is not “random tweaking,” but rather “a controlled multi-objective calibration problem.”

10.8 Ramped and decaying auxiliary losses

To make this calibration less brittle, the notebook does not apply the auxiliary terms at full strength from the beginning of training. Instead it uses an epoch-dependent ramp

$$s_{\text{aux}}(e) = \begin{cases} 0, & e \leq e_{\text{start}}, \\ \frac{e - e_{\text{start}}}{e_{\text{full}} - e_{\text{start}}}, & e_{\text{start}} < e < e_{\text{full}}, \\ 1, & e_{\text{full}} \leq e \leq e_{\text{decay}}, \\ s_{\text{final}}, & e \geq e_{\text{decay}}, \end{cases} \quad (152)$$

where e is the current epoch in the simplified description. The current notebook generalizes this slightly further by interpolating linearly from 1 down to s_{final} between e_{full} and e_{decay} . Concretely, the implemented schedule is

$$s_{\text{aux}}(e) = \begin{cases} 0, & e \leq e_{\text{start}}, \\ \frac{e - e_{\text{start}}}{e_{\text{full}} - e_{\text{start}}}, & e_{\text{start}} < e < e_{\text{full}}, \\ 1 - \frac{e - e_{\text{full}}}{e_{\text{decay}} - e_{\text{full}}}(1 - s_{\text{final}}), & e_{\text{full}} \leq e < e_{\text{decay}}, \\ s_{\text{final}}, & e \geq e_{\text{decay}}. \end{cases} \quad (153)$$

The effective training objective is therefore

$$L_{\text{total}}(e) = L_v + \lambda_{x_0} L_{x_0} + s_{\text{aux}}(e) (\lambda_{\text{lf},T} L_{\text{lf},T} + \lambda_{\text{var}} L_{\text{var}}), \quad (154)$$

because in the current notebook version the x_0 -reconstruction term remains active from the start, while the more heuristic spectral-calibration terms are the ones that are ramped in.

The purpose is to let the network first learn the main transport field from L_v , and only later use the auxiliary terms to calibrate endpoint reconstruction, large-scale temperature power, and channel variances. This is more coherent than imposing all objectives at full strength from epoch 0, because it reduces the chance that calibration terms distort the early learning of the transport itself.

The later decay is equally important. Once the generated spectra have reached the correct amplitude, continuing to push low-frequency T power and smoothed variances with maximal strength can overshoot the target. The decay stage therefore turns the auxiliary losses from “strong corrective forces” into “milder regularizers” near the end of training.

11 Timestep Sampling Bias

The notebook does not sample training times uniformly in the raw variable u . Instead it defines

$$\tau = \begin{cases} u, & p = 1, \\ 1 - (1 - u)^p, & p \neq 1, \end{cases} \quad u \sim \text{Uniform}(0, 1), \quad (155)$$

and then

$$t = t_{\min} + (t_{\max} - t_{\min})\tau. \quad (156)$$

Interpretation:

- $p = 1$: uniform in t ,
- $p > 1$: bias toward high t ,
- $0 < p < 1$: bias toward low t .

The current active run uses

$$p = 0.6, \quad (157)$$

because low- t errors were found to matter most for final sample amplitude in the linear flow setup.

12 Optimization and EMA

The trainer uses:

- optimizer: Adam,
- batch size: 250,
- gradient clipping: $\|\nabla\|_2 \leq 1$,
- base learning rate: 2×10^{-5} ,
- a flat LR hold for 300 epochs,
- then cosine decay to 10% of the base LR,
- exponential moving average of weights with decay 0.99,
- EMA used for evaluation only after a warmup of 300 epochs.

This EMA warmup was added because a slower EMA (historically 0.999) lagged too far behind the model in short/medium runs and artificially flattened early spectral diagnostics.

13 Flat-Sky Spectral Diagnostics

13.1 Power spectrum estimator

For each 64×64 patch at angular pixel size r (in radians), let the physical side lengths be

$$L_x = Wr, \quad L_y = Hr. \quad (158)$$

The notebook computes discrete Fourier transforms on the grid

$$\ell_x = 2\pi\nu_x, \quad \ell_y = 2\pi\nu_y, \quad (159)$$

where ν_x, ν_y are the discrete FFT frequencies returned by `fftfreq` with spacing r . The radial multipole assigned to each Fourier pixel is

$$\ell = \sqrt{\ell_x^2 + \ell_y^2}. \quad (160)$$

If T_{ij} is the temperature map on the patch, the notebook uses the convention

$$\hat{T}(\ell_x, \ell_y) = \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} T_{ij} e^{-2\pi i(i\nu_y + j\nu_x)}. \quad (161)$$

For T , the flat-sky power is estimated by

$$P^{TT}(\ell_x, \ell_y) = \frac{r^2}{HW} |\hat{T}(\ell_x, \ell_y)|^2. \quad (162)$$

The prefactor $r^2/(HW)$ converts the squared FFT amplitude into an angular power density consistent with the patch pixel area.

For polarization, the notebook performs the standard local flat-sky E/B decomposition [1,2]:

$$E_\ell = -Q_\ell \cos(2\phi_\ell) - U_\ell \sin(2\phi_\ell), \quad (163)$$

$$B_\ell = Q_\ell \sin(2\phi_\ell) - U_\ell \cos(2\phi_\ell), \quad (164)$$

where

$$\phi_\ell = \text{atan2}(\ell_y, \ell_x). \quad (165)$$

These are the flat-sky analogues of the full-sky spin-2 $Q/U \leftrightarrow E/B$ transform. The corresponding power estimators are then

$$P^{EE} = \frac{r^2}{HW} |E_\ell|^2, \quad P^{BB} = \frac{r^2}{HW} |B_\ell|^2, \quad P^{TE} = \frac{r^2}{HW} \Re(T_\ell E_\ell^*). \quad (166)$$

The reason the notebook averages over annuli is the same reason that the full-sky formalism averages over m at fixed ℓ : under statistical isotropy, the ensemble covariance depends only on the magnitude of the wavevector, not on its orientation. So the annular average

$$\hat{C}_b^{XY} = \frac{1}{|\mathcal{I}_b|} \sum_{\ell \in \mathcal{I}_b} P^{XY}(\ell) \quad (167)$$

is the discrete flat-sky estimator of the radial spectrum C_ℓ^{XY} .

One can also see why the plotted D_ℓ -type quantity is natural on the flat sky. In the continuum plane, the variance of a homogeneous scalar field is

$$\text{Var}[T(\mathbf{x})] = \int \frac{d^2\ell}{(2\pi)^2} C_\ell^{TT, \text{flat}} = \int_0^\infty \frac{\ell d\ell}{2\pi} C_\ell^{TT, \text{flat}}. \quad (168)$$

Writing $\ell d\ell = \ell^2 d \ln \ell$ shows again that $\ell^2 C_\ell/(2\pi)$, and hence approximately $\ell(\ell+1)C_\ell/(2\pi)$, is the contribution to variance per logarithmic radial-frequency interval.

The radial spectra are formed by azimuthal averaging in up to 30 ℓ -bins. If b is a bin and $\mathcal{I}_b$ is the set of Fourier pixels whose radial multipole falls into that bin, then for a single patch

$$\hat{C}_b^{XY} = \frac{1}{|\mathcal{I}_b|} \sum_{\ell \in \mathcal{I}_b} P^{XY}(\ell), \quad XY \in \{TT, EE, BB, TE\}. \quad (169)$$

The displayed curve is then an average over many held-out or generated patches:

$$\bar{C}_b^{XY} = \frac{1}{N_{\text{patch}}} \sum_{n=1}^{N_{\text{patch}}} \hat{C}_{b,n}^{XY}. \quad (170)$$

Finally, the plotted quantity is

$$\hat{D}_b^{XY} = \frac{\ell_b(\ell_b + 1)}{2\pi} \overline{C}_b^{XY}, \quad (171)$$

with ℓ_b the bin center. On small patches and for $\ell \gg 1$, this is essentially the flat-sky analogue of the standard full-sky D_ℓ .

13.2 Smoothing before spectra

Before spectral estimation, the notebook applies the same isotropic Gaussian blur to both reference and generated patches:

$$x \mapsto G_{\sigma_{\text{spec}}} * x, \quad \sigma_{\text{spec}} = 1.2 \text{ pixels}. \quad (172)$$

This was introduced to suppress high- ℓ discretization and solver artefacts on small patches. In Fourier space, this smoothing multiplies each mode approximately by

$$\exp\left[-\frac{1}{2}\sigma_{\text{ang}}^2\ell^2\right], \quad (173)$$

where $\sigma_{\text{ang}} = \sigma_{\text{spec}} r$ in angular units. Thus it acts as a controlled low-pass filter before the power estimator is applied.

13.3 Why x_0 -hat spectra are used

An important debugging fix was to stop computing spectra from the raw terminal sampler state x_t . The correct object to compare to the data prior is the denoised estimate

$$\hat{z} = \hat{x}_0. \quad (174)$$

Using raw x_t at $t < 1$ leaves residual Gaussian noise in the map and artificially pushes spectral power toward high ℓ , which previously mimicked a “shifted” TT peak.

13.4 Variance-matched diagnostic

The post-training spectral block also constructs a purely diagnostic, post hoc variance-matched version of the generated maps:

$$s_c = \frac{\text{std}(\text{reference}_c)}{\text{std}(\text{generated}_c)}, \quad \hat{x}_c^{\text{vm}} = s_c \hat{x}_c. \quad (175)$$

This is *not* part of sampling itself. It is used only to answer the question:

If the generated maps were corrected by a simple per-channel multiplicative variance rescaling, how much of the spectral discrepancy would disappear?

The observation that variance-matched spectra improve a lot was one of the main motivations for adding L_{var} .

If the generated fields were exactly multiplicatively miscalibrated,

$$\hat{T} \approx s_T T, \quad \hat{E} \approx s_E E, \quad \hat{B} \approx s_B B, \quad (176)$$

then the spectra would transform approximately as

$$\hat{C}_\ell^{TT} \approx s_T^2 C_\ell^{TT}, \quad \hat{C}_\ell^{EE} \approx s_E^2 C_\ell^{EE}, \quad \hat{C}_\ell^{BB} \approx s_B^2 C_\ell^{BB}, \quad \hat{C}_\ell^{TE} \approx s_T s_E C_\ell^{TE}. \quad (177)$$

The fact that post hoc variance matching helps substantially is therefore strong evidence that the remaining error is dominated by under-dispersion rather than by a wrong acoustic scale or by catastrophic mode mixing.

14 Diagnostic Quantities Added During Debugging

The notebook now records and/or prints:

- `clip_frac`: fraction of sampler-state entries that hit the state clamp,
- `val_loss_field`: vector-field validation MSE,
- `val_loss_field_zero`: zero-predictor baseline for the vector-field loss,
- `val_loss_x0`: clean-map reconstruction MSE,
- `val_loss_t_lowfreq`: low-frequency T -channel validation loss,
- `val_loss_var_smoothed`: smoothed variance calibration loss,
- `aux_loss_scale`: current value of the auxiliary-loss ramp,
- timestep-binned validation losses,
- raw and variance-matched TT relative errors,
- per-channel RMS before and after spectral smoothing,
- `spectral_selection_score`: a scalar criterion for model selection,
- `best_spectral_score_so_far`: running best value of that criterion.

These diagnostics were added to separate several failure modes that looked similar visually but were mathematically different:

1. true sampler blow-up,
2. wrong endpoint quantity (x_t instead of $\hat{x}_0$),
3. poor vector-field learning,
4. under-dispersion / variance shrinkage,
5. low-frequency under-power in T .

15 Chronology of Major Fixes and What They Solved

15.1 Fix 1: true validation split

Originally, the notebook compared against training data while labeling it as validation. This was corrected so that `val_norm` is used for held-out diagnostics and spectral references whenever available.

15.2 Fix 2: clip fraction and endpoint range

Early score-based samples saturated at the state clamp, so `clip_frac` was introduced to quantify instability rather than hiding it in the spectra.

15.3 Fix 3: denoise before spectra

The notebook was corrected to compare spectra of $\hat{x}_0$ instead of raw terminal x_t . This removed an artificial high- ℓ bias.

15.4 Fix 4: direct flow matching

Because the original score-conversion sampler remained unstable even after endpoint handling changes, the notebook switched from epsilon/score prediction to direct vector-field prediction and Heun ODE sampling.

15.5 Fix 5: EMA for sampling, then EMA warmup

EMA was added for better sampling stability, but a too-slow EMA initially made early diagnostics misleadingly flat. This led to the current 0.99 EMA with warmup before evaluation.

15.6 Fix 6: low- t bias and x_0 reconstruction loss

After instability was removed, the main issue became underpowered spectra. Low- t bias and an auxiliary x_0 reconstruction loss were added because endpoint errors in the linear path scale like $(1 - t)$.

15.7 Fix 7: low-frequency T loss

When the TT peak shape became roughly correct but low/mid- ℓ power was still suppressed, the notebook added $L_{\text{lf},T}$.

15.8 Fix 8: smoothed variance matching loss

When variance-matched spectra looked much better than raw spectra, L_{var} was added to explicitly calibrate smoothed-domain channel variances during training.

15.9 Fix 9: staged calibration and spectral checkpoint selection

Once the spectra could move from too low to too high without changing their basic shape, it became clear that the remaining task was amplitude calibration rather than structural correction. The notebook therefore introduced two additional ideas:

1. ramp the auxiliary losses in over epochs rather than applying them fully from the start,
2. select checkpoints by a dedicated spectral validation score rather than by the final epoch or by L_v alone.

16 Current Active Configuration

The current active training cell uses:

Quantity	Value
Prior used for training	true CMB prior <code>prior_f</code>
Patch size	64×64
Patch resolution	6.7' per pixel
Path	linear, $\alpha_t = t$, $\beta_t = 1 - t$
Sampling domain	$t \in [0, 1]$
Sampler	Heun ODE ($\sigma = 0$)
Spectral smoothing	1.2 pixels
Spectral ℓ -cut	$\ell \leq 1200$
State clamp	± 5 (diagnostic guardrail)
Network channels	[32, 64, 128, 256]
Residual layers per stage	2
Time embedding dimension	64
Epochs	2500
Batch size	250
Base learning rate	2×10^{-5}
LR hold	300 epochs
Cosine floor factor	0.1
EMA decay	0.99
EMA eval warmup	300 epochs
t -bias power	0.6
λ_{x_0}	0.25
$\lambda_{\text{lf}, T}$	0.5
$\sigma_{\text{lf}, T}$	(2.0, 4.0) pixels
λ_{var}	1.0
σ_{var}	1.2 pixels
Auxiliary ramp start	epoch 50
Auxiliary ramp full strength	epoch 200
Auxiliary ramp decay epoch	epoch 500
Auxiliary final scale	0.55
Spectral checkpoint cadence	every 50 epochs until 500, then every 250

17 Discussion: Interpretation of the Current Remaining Error

At the latest stage, the dominant error is no longer catastrophic instability. Instead:

- the spectral peak locations are broadly correct,
- clipping is near zero,
- but the raw generated spectra can be either lower or higher than the validation spectra,
- while variance-matched spectra often improve significantly.

This indicates that the current model is much closer to the correct spatial structure than before, but still has a nontrivial calibration problem in amplitude. In other words, the remaining gap is not mainly “where the peak should be” but “how strong the generated modes should be.”

17.1 A more systematic way to “hit the right spot”

The notebook now adopts the following point of view:

1. L_v is the core transport-learning objective,
2. the other losses are calibration terms,
3. the model should therefore be selected by a validation criterion that measures the actual spectral target.

To make this explicit, the notebook constructs a scalar spectral selection score combining shape mismatch and amplitude mismatch. Schematically, if

$$\mathcal{E}_{\text{shape}} = \sqrt{\frac{1}{|\mathcal{S}|} \sum_{XY \in \mathcal{S}} \frac{1}{B} \sum_{b=1}^B \left(\log \hat{D}_{b,\text{gen}}^{XY} - \log \hat{D}_{b,\text{ref}}^{XY} \right)^2}, \quad (178)$$

and if

$$\mathcal{E}_{\text{amp}} = \sqrt{\frac{1}{3} \sum_{c \in \{T, Q, U\}} \left(\log \sigma_c^{\text{gen}} - \log \sigma_c^{\text{ref}} \right)^2}, \quad (179)$$

then the notebook records a scalar quantity of the form

$$J_{\text{spec}} = \mathcal{E}_{\text{shape}} + \mathcal{E}_{\text{amp}}. \quad (180)$$

This criterion is intentionally simple: it penalizes wrong spectral shape and wrong global amplitude in a common logarithmic geometry. The best checkpoint is then the one with the smallest J_{spec} , rather than necessarily the one from the last epoch.

This is scientifically and algorithmically useful because it separates two questions that were previously conflated:

- “Has the network learned the transport field at all?”
- “Among the plausible transport fields, which checkpoint gives the best generated spectra?”

17.2 How to interpret future overshoot or undershoot

With this framework, future tuning becomes more interpretable:

- if only low/mid- ℓ TT overshoots while other channels remain reasonable, then $\lambda_{\text{lf},T}$ is likely too strong;
- if TT, EE, BB all move high or low together, then the issue is more likely in the variance-calibration term λ_{var} ;
- if the spectral shape itself degrades, then the model is no longer just miscalibrated: the base transport learning or optimizer setup may need revision.

So the notebook is no longer being tuned purely by eye. The intended strategy is:

1. learn the transport with L_v ,
2. calibrate with gradually introduced auxiliary losses,
3. and choose checkpoints by J_{spec} .

18 Conclusion

In summary, the notebook has evolved from a classical epsilon-prediction diffusion prototype into a more stable and better-targeted linear flow-matching prior model for normalized T, Q, U CMB patches. The key stages were:

1. build realistic full-sky simulated priors and convert them to flat patches;
2. normalize train/validation splits channel-wise from train statistics;
3. define a Gaussian conditional probability path $x_t = \alpha_t z + \beta_t \varepsilon$;
4. first try score-based diffusion training and sampling;
5. diagnose stiff reverse dynamics and clipping;
6. switch to direct vector-field prediction and Heun ODE sampling;
7. add losses and diagnostics to improve endpoint reconstruction, low-frequency TT structure, and variance calibration.

This structure is scientifically useful because it incrementally moves the notebook from “a model that trains” to “a model whose generated prior samples are spectrally and statistically faithful enough to be used inside a posterior sampling pipeline.”

More broadly, the notebook illustrates a recurring lesson in scientific machine learning: once the gross numerical instabilities are removed, the remaining challenge is often not discovering a completely different model but calibrating a mathematically principled model against the observable that matters scientifically. Here that observable is not merely a training MSE, but the family of flat-sky spectra TT, EE, BB, TE computed in the same normalized patch domain in which the model is trained.

The practical consequence is that the current notebook should be viewed as a methods prototype with three essential ingredients already in place:

1. a physically interpretable prior-construction pipeline from full-sky harmonic simulations to masked gnomonic T, Q, U patches;
2. a stable continuous-time generative model based on direct flow matching rather than an unstable score conversion;
3. a validation framework that separates structural spectral errors from pure amplitude-calibration errors.

These ingredients are sufficient to support the next scientific step: embedding the learned prior into a posterior sampler for component separation and uncertainty quantification.

References

- [1] M. Zaldarriaga and U. Seljak, “All-sky analysis of polarization in the microwave background,” *Physical Review D* **55** (1997), 1830–1840.
- [2] M. Kamionkowski, A. Kosowsky, and A. Stebbins, “Statistics of cosmic microwave background polarization,” *Physical Review D* **55** (1997), 7368–7388.
- [3] K. M. Górski, E. Hivon, A. J. Banday, B. D. Wandelt, F. K. Hansen, M. Reinecke, and M. Bartelmann, “HEALPix: A Framework for High-Resolution Discretization and Fast Analysis of Data Distributed on the Sphere,” *The Astrophysical Journal* **622** (2005), 759–771.

- [4] A. Lewis, A. Challinor, and A. Lasenby, “Efficient Computation of Cosmic Microwave Background Anisotropies in Closed Friedmann–Robertson–Walker Models,” *The Astrophysical Journal* **538** (2000), 473–476.
- [5] N. Aghanim et al. (Planck Collaboration), “Planck 2018 results. I. Overview and the cosmological legacy of Planck,” *Astronomy & Astrophysics* **641** (2020), A1.
- [6] B. Thorne, J. Dunkley, D. Alonso, and S. Næss, “The Python Sky Model: software for simulating the Galactic microwave sky,” *Monthly Notices of the Royal Astronomical Society* **469** (2017), 2821–2833.
- [7] A. Zonca, T. A. J. Louis, B. Thorne, N. Krachmalnicoff, and J. Borrill, “PySM 3: A Python Sky Model for simulation of Galactic microwave emission,” software/documentation release, 2021.
- [8] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems 33* (2020).
- [9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,” in *International Conference on Learning Representations* (2021).
- [10] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” in *International Conference on Learning Representations* (2023).